



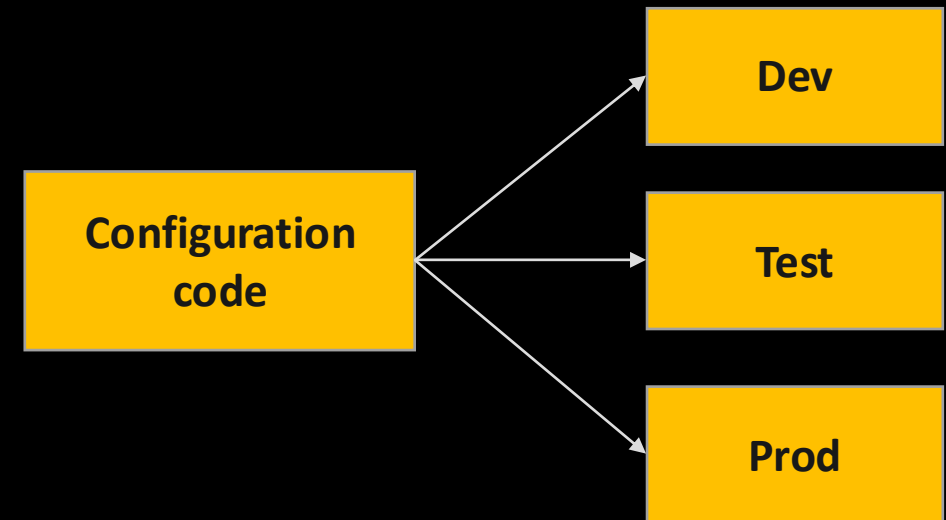
Puppet



What is configuration management



- Configuration management is a systems engineering process for establishing and maintaining consistency of a product's performance, functional and physical attributes with its requirements, design and operational information throughout its life
- These tools will help to create, configure the infrastructure by using a central server
- It is used for infrastructure as-a code



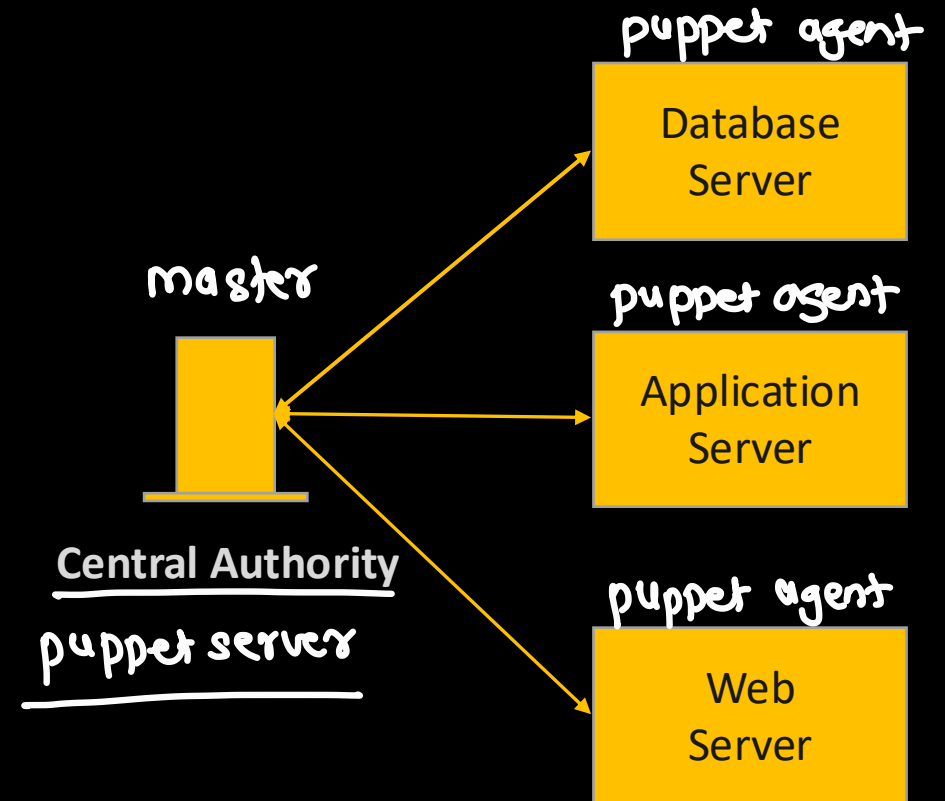
Types



- CM tools tend to implement one (or both) of these models of management
- There are two ways
 - Pull based model
 - Push based model

Pull based model

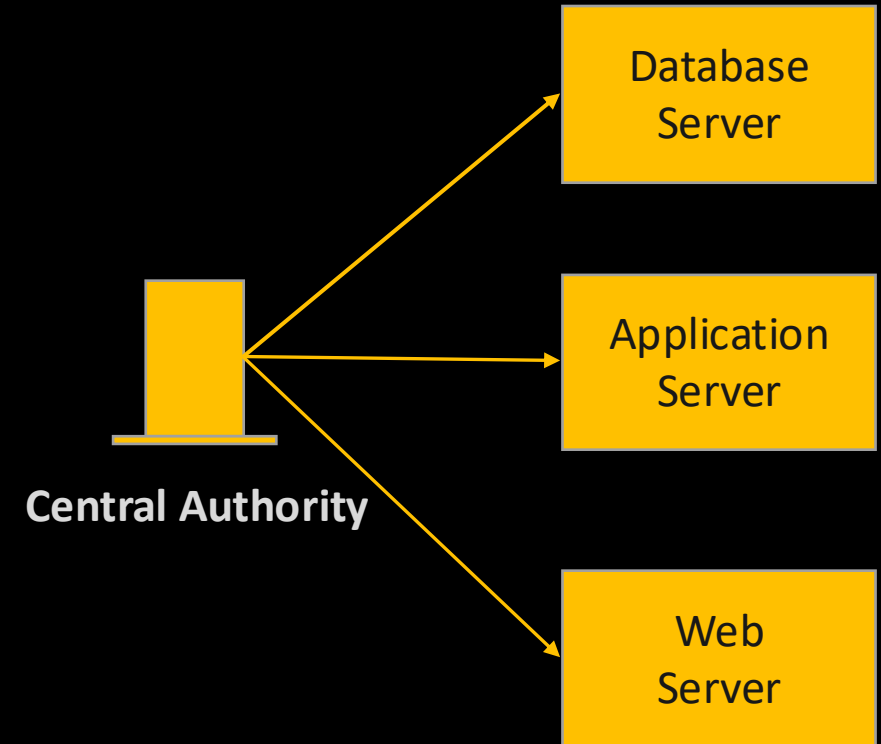
- The server being provisioned runs an agent that asks a central authority if and when it has any updates that it should run
- The server here is known as a node
- The agent runs as a daemon
- These types of tools require an agent to be installed on all the machines and a central authority to be setup
- E.g.
 - Puppet



Push based model



- The server being provisioned runs pushes any updates to the other machines
- E.g.
 - Ansible



Idempotency

→ configuration file

- The ability to repeatedly apply a manifest to guarantee a desired resource state on a system, with the same results every time
- If the given resource is already in the desired state, Puppet performs no action
- If the given resource is not in the desired state, Puppet takes whatever action is necessary to put the resource into the desired state

```
#!/bin/bash
```

```
sudo apt-get install -y apache2
```

non idempotent

```
package { 'apache2':  
    ensure => 'present'  
}
```

idempotent

Resource



- Puppet comes with built-in resources which have their own resource types
- Every resource is of a certain resource type

```
type { 'title':  
  arg => 'value'  
}
```

resource

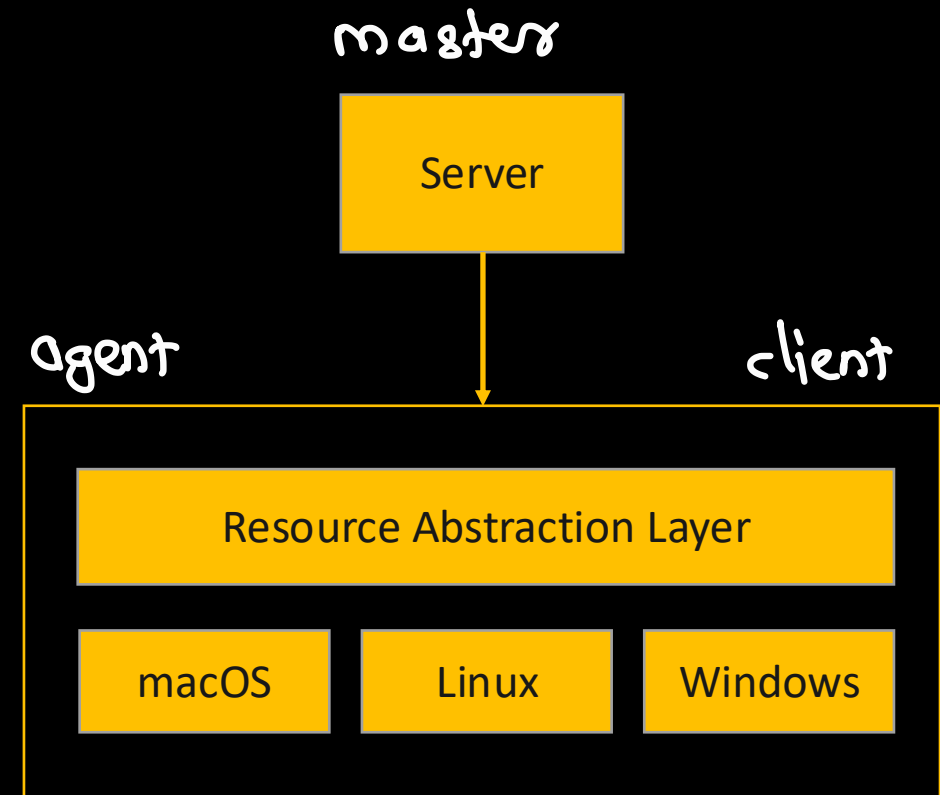
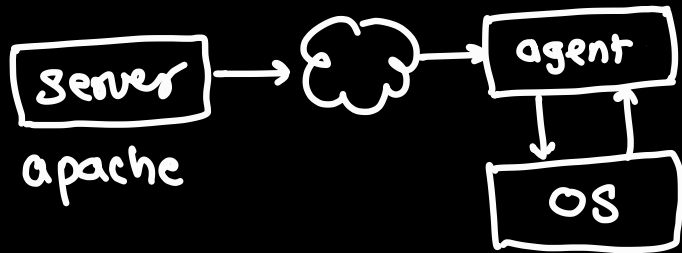


```
package { 'apache2':  
  ensure => 'present'  
}
```

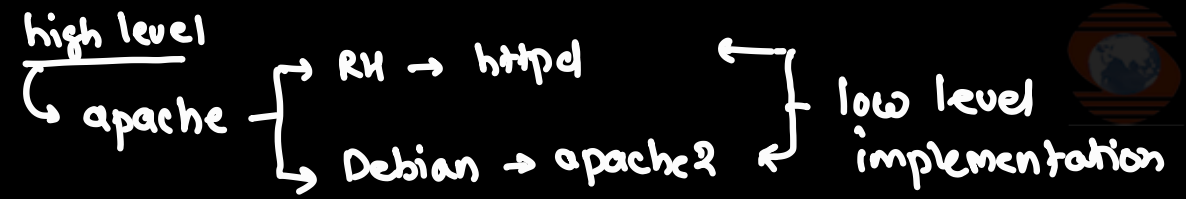
puppet language syntax

Resource Abstraction Layer (RAL)

- It is a key feature of Puppet, where it separates out the resources from their implementations
- It provides the consistent way to interact with the base OS
- The platform specific configurations exist from providers



How does it work ?



Puppet Code → will be kept on the server

- The high-level of the desired end state we have for our servers

Puppet Code

Resource Abstraction Layer

- What allows us to use resources to write the puppet code

Resource Abstraction Layer

Providers

- Part of RAL that takes high-level code and converts it to the way managed system can use it

Providers

Puppet Flavors

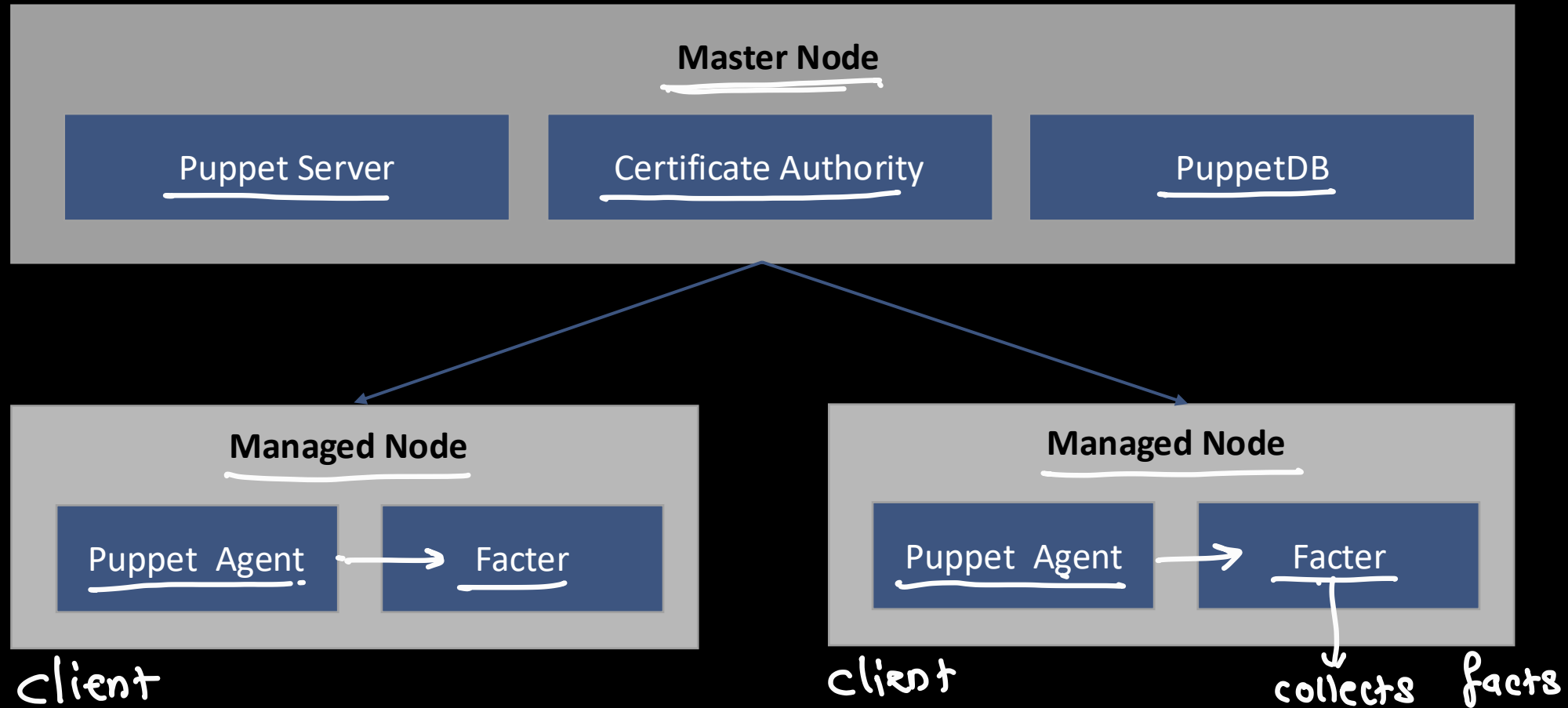


- There are two flavors of Puppet
 - Enterprise Puppet → paid
 - Open Source Puppet → community (free)

Architecture



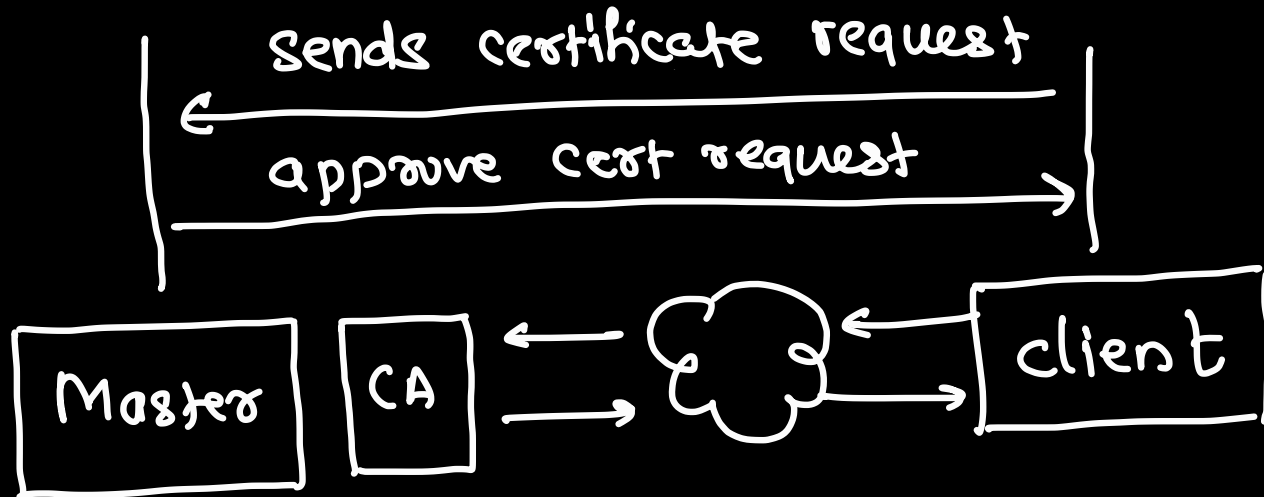
server → puppet master



Puppet Master



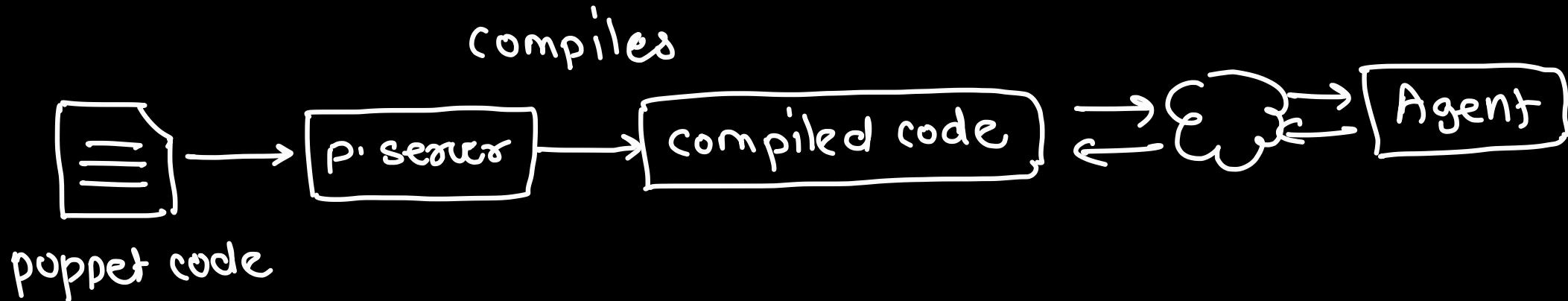
- All Puppet masters are comprised of the Puppet Server and certification authority
- The Puppet Server is a Java Virtual Machine and compiles Puppet code for use on the agents
- The certificate authority provides certificates so that master and agents can communicate safely
- Optional PuppetDB (automatically added in Enterprise) can be added, which collects Puppet-generated data and enables some advanced features
- PuppetDB can also be accessed by other services that may need Puppet data.



Puppet Agent

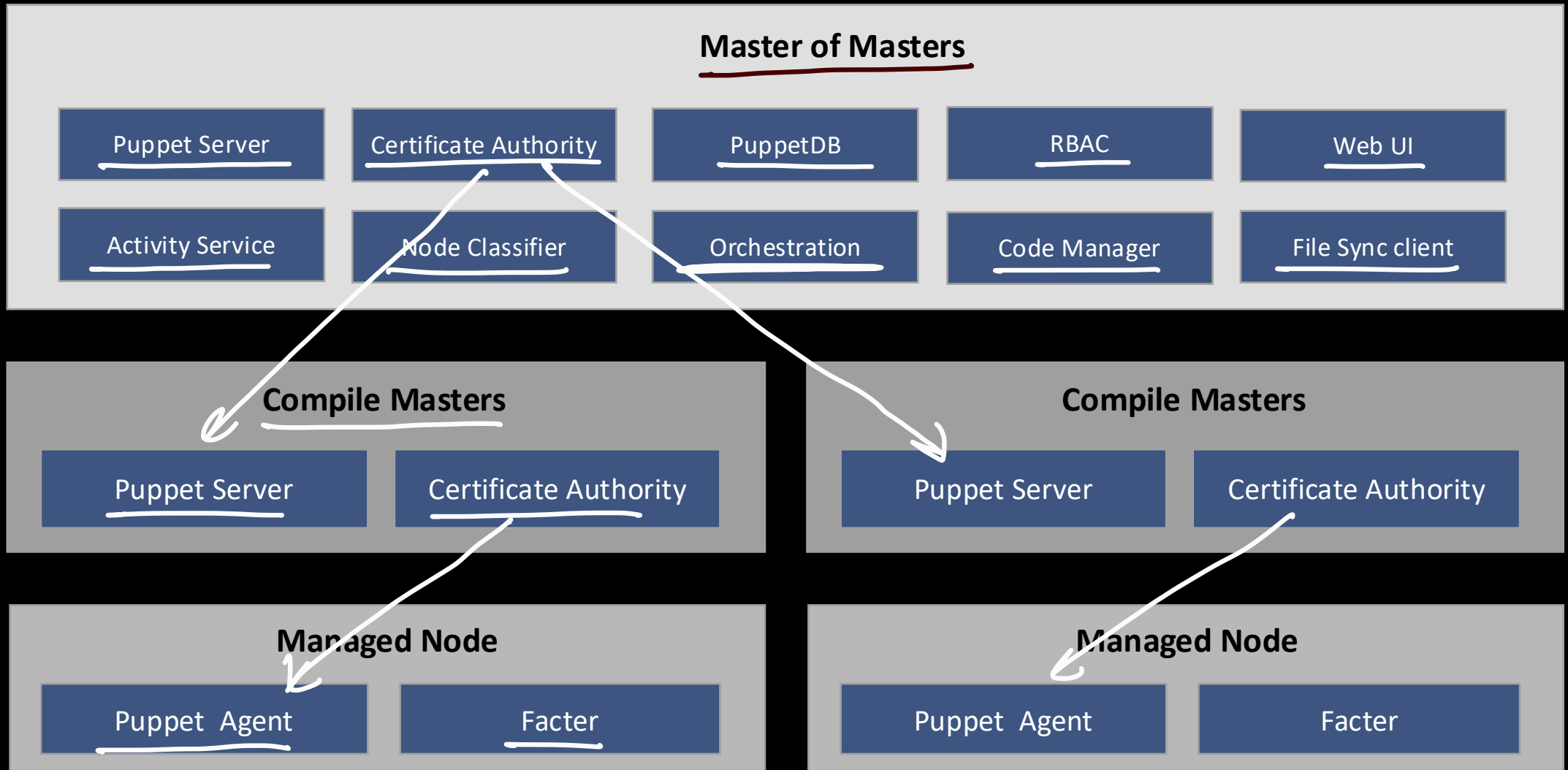


- The Puppet Agent application needs to be on all nodes we manage with Puppet
- The Puppet agent runs as a background service and periodically queries for compiled Puppet code from the server to enforce change with
- Puppet agents also contain Facter, which records facts about our system that are then passed to the master. We can use these facts in our Puppet code.



Architecture

master of master nodes = Enterprise



Master of Masters

PE → Puppet Enterprise



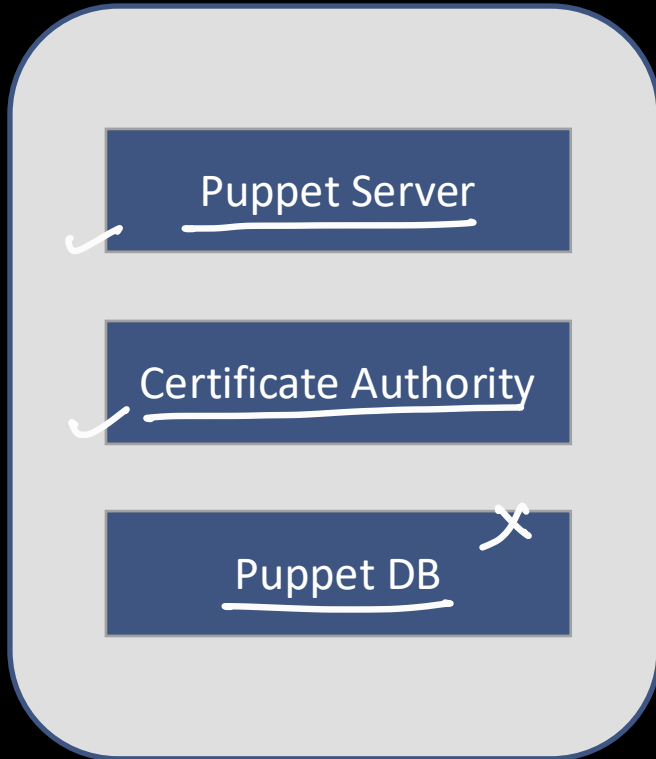
- PE adds the concept of a "Master of Masters" — or a Puppet master that contains all the additional features of PE
- This includes
 - the Puppet server
 - Certificate authority
 - Role-based access control
 - Orchestration for multi-tiered applications
 - A node classifier that lets us assign code to managed systems
 - A web-based console that lets us manage much of the previously-mentioned tools
- The Master of Masters also contains Code Manager and a file sync client, which are used to ensure the Puppet code is consistent across a multi-master setup.

The Lifecycle of a Puppet Run



apache

Master Node



Agent node sends facts to the master and requests a catalog



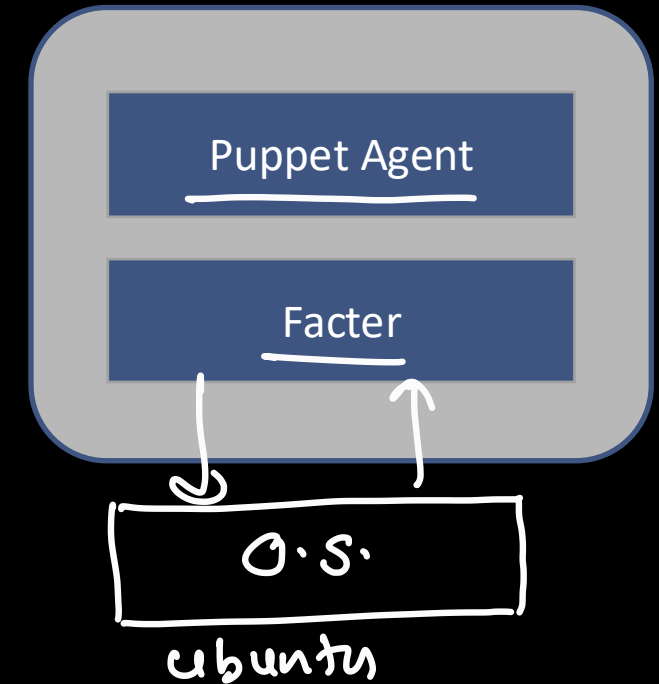
Master compiles and sends the node's catalog using the resources of information the master has an access to



Agent sends a report back to the master



Managed Node





Open Source Puppet

Installation and configuration



- System Requirements
 - For nodes < 10
 - #Cores: 2
 - RAM : 1GB
 - For nodes < 1000
 - #Cores: 1
 - RAM : 4GB
- Add puppet in /etc/hosts file
 - `sudo vim /etc/hosts`
 - `127.0.0.1 puppet localhost`

Installation and configuration



- Add the puppet apt source
 - > `wget https://apt.puppetlabs.com/puppet6-release-bionic.deb`
 - > `sudo dpkg -i puppet6-release-bionic.deb`
 - > `sudo apt update`
- Install puppetserver package
 - > `sudo apt-get install puppetserver`
- Update memory configuration
 - > `sudo vim /etc/default/puppetserver`
- Start and enable the puppet server
 - > `sudo systemctl start puppetserver`
 - > `sudo systemctl enable puppetserver`

Puppet Agent



- Add the puppet apt source
 - > `wget https://apt.puppetlabs.com/puppet6-release-bionic.deb`
 - > `sudo dpkg -i puppet6-release-bionic.deb`
 - > `sudo apt update`

- Install puppetserver package
 - > `sudo apt-get install puppet-agent`

- Start and enable the puppet server
 - > `sudo systemctl start puppet`
 - > `sudo systemctl enable puppet`

- Check if configuration works
 - > `sudo puppet agent --fingerprint`

Certificate Authority



- To see our full list of certs
 - > sudo puppetserver ca list --all
- Accept node into our Puppet infrastructure
 - > sudo puppetserver ca sign --certname <name>
- To setup external CA
 - > sudo puppetserver ca setup
- To generate new certificate
 - > sudo puppetserver ca generate
- To import external CA authority
 - > sudo puppetserver import
- To revoke a certificate
 - > sudo puppetserver revoke --certname <name>
- To revoke and remove associated files
 - > sudo puppetserver clean --certname <name>



Puppet Language

Puppet Forge



- To configure any of our nodes, we need to use a Puppet module
- Module is a collection of code set to configure a single specific overall item, such as Apache, nginx etc
- The Puppet Domain-Specific Language used to write these modules
- The Puppet Forge is a PuppetLabs-hosted repository of Puppet modules, both user-submitted and provided by PuppetLabs itself

Puppet Forge - Labels



- **Supported**

- A supported module is one written by PuppetLabs that works on both enterprise and open source versions
- These modules are actively maintained and tested against Puppet Enterprise to ensure the highest quality

- **Partner**

- A partner module is one written by a "Puppet partner"
- these are all modules written by the organization that produces the tool it configures
- These modules are also rigorously tested against Puppet Enterprise, actively maintained, and are held to the highest standards.

- **Approved**

- An approved module is a user-submitted module that's "well-written" and "reliable".
- These modules follow best practices and are actively maintained, although not as stringently tested as supported and partner modules.

- **Tasks**

- Modules with the tasks tag provided task support
- they supply configuration options that can be run outside of the configuration management system.

- **PDK**

- Any module that supports validation testing through the Puppet Development Kit uses the PDK label

Module Structure



- **data and hiera.yaml**
 - Hiera is Puppet's data store, with the hiera.yaml file being where we store our module-specific configuration file and the data directory where we store our actual data
- **examples**
 - This directory stores examples regarding the module's use; for ntp, this shows us how to call the class in our main manifest
- **locales and readmes**
 - Stores localization data for other languages
- **manifests**
 - Our configurations are stored in this directory; covered in detail in the next lesson
- **spec**
 - Stores integration tests for our Puppet code written in RSpec

Module Structure



- **templates and files**
 - Stores files to be distributed to nodes; templates specifically stores template files that need to be rendered before being added to a node
- **types**
 - Stores aliases for custom resource types
- **tasks**
 - Stores tasks that can be used outside of configuration management
- **lib**
 - Stores plugins, including things like additional resource types and custom facts
- **facts.d**
 - Stores additional facts

Manifest files



- **Class**
 - A named section of code with configuration instructions
- **Resource**
 - A single stanza that sets parameters for a resource type
- **Resource Type**
 - Handles the configuration of our system through a set of defined parameters we supply

Manifest files



```
class ntp::install {  
  
    if $ntp::package_manage {  
  
        package { $ntp::package_name:  
                  ensure => $ntp::package_ensure,  
                }  
  
    }  
  
}
```

Main Manifest



- To use any module, it's not good enough to simply write a bunch of classes
- We need to assign our overall module to whichever nodes it needs to configure, as well
- This can be done in two ways
 - via a Puppetfile
 - using a main manifest